Identity on Demand

Multi-head Gated Value Residuals

Leon Lehmann

Empero AI Research lehmann@empero.org Correspondence: research@empero.org

Code: github.com/empero-org/mgvr

Abstract

A Transformer layer that needs a token's identity, to copy it or to look something up with it, has to recover it from a residual stream that every earlier layer has written into. Value residual learning hands every layer a fixed share of the first layer's values instead. We make that share a decision. *Multi-head Gated Value Residuals* (MGVR) let each layer compute from its own input, for every token and every key–value head, a gate on its values and a mix of the first layer's value heads, so that each head draws as much token identity as it needs, when it needs it. MGVR adds 0.2% of the active parameters, leaves the attention kernel and the size of the key–value cache unchanged, and starts exactly as a value-residual Transformer. In a controlled pretraining comparison over 1B tokens, MGVR reaches the Transformer's held-out loss with 16–22% fewer steps from step 200 onwards, worth 1.15–1.23× effective compute after its 4.3% step-time overhead, and ends 0.050 nats lower. In a synthetic study of six variants with shared data and initialisation over three seeds, MGVR forms induction heads faster at every learning rate, cutting the time from the attention pattern to accurate copying by 40%, and learns two-hop associative recall 25–49% faster than a Transformer and 7–18% faster than a static value residual. Ablations trace the composition gains to the token-dependent mixing of value heads.

KEY FINDINGS

- **The same loss in 16–22% fewer steps.** In a controlled 1B-token pretraining comparison, MGVR reaches the Transformer's held-out loss with 16–22% fewer steps at every evaluation from step 200 onwards. Charged its 4.3% longer step time, that is 1.15–1.23× effective compute. It ends 0.050 nats lower (2.812 vs 2.862).
- **Copying works sooner.** Induction heads turn into accurate copiers earlier at every learning rate. The time from the induction attention pattern to 90% copy accuracy falls by 40% (133 to 80 steps).
- **Composition is learnt faster.** On two-hop associative recall, MGVR learns 25% and 49% faster than the Transformer and 7% and 18% faster than a static value residual, with a four to ten times smaller spread across seeds.
- **Token-dependent mixing does the work.** Mixing layer 0's value heads per token, on its own, matches full MGVR on composition. The method adds 0.2% of the active parameters and drops into any multi-head or grouped-query attention layer.

1 Introduction

A Transformer (Vaswani et al., 2017) reads each token's identity once, at its embedding. A later layer that needs it has to recover it from the residual stream, which by then holds everything the earlier layers have written. Such layers are common: an induction head copies the token it attends to (Elhage et al., 2021; Olsson et al., 2022), and a lookup retrieves the value stored under a key. Attention does not make

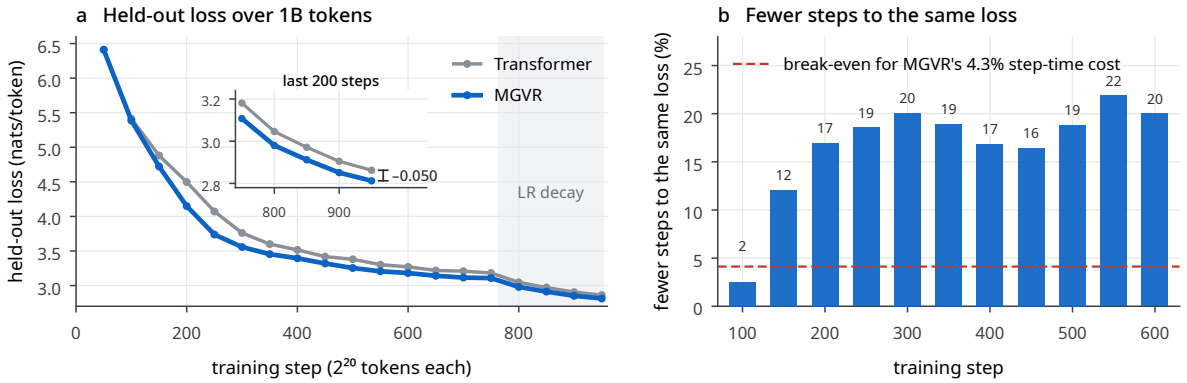


Figure 1 **MGVR reaches the same loss in fewer steps.** **a** Held-out loss of a Transformer and of the same model with MGVR, trained from identical initial weights on identical data. The inset magnifies the learning-rate decay. **b** The fraction of steps MGVR saves to reach the Transformer's loss, read off the Transformer's stable-phase curve. Above the dashed line, MGVR also wins in wall-clock time despite its 4.3% longer steps. From step 650 on, MGVR is below the best loss the Transformer reaches before its decay.

the job easier. A head's output is a weighted sum of the values of the positions it attends to, and those values are computed from the same crowded residual streams, at the head's own depth.

Value residual learning (Zhou et al., 2024) offers a shortcut. Every layer adds the first layer's values to its own, $\tilde{v}^{(\ell)} = v^{(\ell)} + \lambda\, v^{(0)}$. Attention is linear in the values, so every head then reads, at every position it attends to, a fixed share of that position's first-layer value: its identity, as the first layer encoded it. The residual stream is untouched. The NanoGPT speedrun (Jordan et al., 2024a) adopted the idea in the form of extra embeddings mixed into the values of its attention layers. In both forms the share is the same for every head and every token. A head that copies tokens and a head that aggregates context receive the same dose of identity, whether or not the token at hand calls for it.

We make the dose a decision. *Multi-head Gated Value Residuals* (MGVR) let every layer compute, from the same normalised input its attention reads, a gate on each of its key–value heads and a mixing row over the first layer's value heads, separately for every token (Fig. 2). A head can draw much identity or none, or a different combination of identity features, and the choice changes from token to token: identity on demand. MGVR is initialised to reproduce a value-residual Transformer exactly, and a plain Transformer exactly when the residual weight is zero. It adds 0.2% of the active parameters and touches nothing but the values, so attention kernels, positional encodings and the size of the key–value cache are unchanged.

We test MGVR in two ways. A controlled pretraining comparison measures what it buys. With identical initial weights, data order and hyperparameters, MGVR reaches the Transformer's held-out loss with 16–22% fewer steps throughout training and ends 0.050 nats lower (Fig. 1). A synthetic study measures why. MGVR speeds up the circuits that need token identity, namely the copying step of induction heads and the composition of two lookups, and ablations attribute the gains to its token-dependent mixing of value heads.

Contributions.

- **MGVR**, a drop-in change to the value path of multi-head and grouped-query attention (Section 3). At initialisation it reduces exactly to a value residual, or to standard attention. A reference implementation takes twenty lines.
- **Efficiency evidence.** In a controlled pretraining comparison over 1B tokens, MGVR needs 16–22% fewer steps for the same held-out loss from step 200 onwards. That is 1.15–1.23× effective compute after its 4.3% step-time cost, and it ends 0.050 nats lower (Section 4).

- **Mechanism evidence.** We compare six variants over three seeds with shared data and initialisation (Section 5). MGVR cuts the time from the induction attention pattern to accurate copying by 40%. It learns two-hop recall 25–49% faster than a Transformer and 7–18% faster than a static value residual.
- **Code and results.** The reference module, the synthetic harness that produced every number in Section 5, and the pretraining curves are at `github.com/empero-org/mgvr`.

2 Background

2.1 Attention reads values

We write $x_s^{(\ell)} \in \mathbb{R}^d$ for the residual stream entering layer $\ell \in \{0, \dots, L-1\}$ at position s, and $\hat{x}_s^{(\ell)}$ for its normalised version, the input of the layer's attention (Zhang and Sennrich, 2019). In grouped-query attention (Ainslie et al., 2023), H query heads share G key–value heads of dimension d_h; multi-head attention is the case $G = H$. Head i, in group $g(i)$, outputs at query position t

$$o_{t,i}^{(\ell)} = \sum_{s \le t} a_{t,s,i}^{(\ell)} \, v_{s,g(i)}^{(\ell)}, \qquad v_{s,g}^{(\ell)} = W_{V,g}^{(\ell)} \, \hat{x}_s^{(\ell)}, \tag{1}$$

where the attention weights a come from the queries and keys. Two properties of Eq. (1) matter here. First, the output is linear in the values, so anything added to v_s reaches every query that attends to s, in proportion to the attention it pays. Second, v_s is computed from $\hat{x}_s^{(\ell)}$, so what a head reads at position s is whatever the residual stream at s holds at depth ℓ. Rotary position embeddings (Su et al., 2021) rotate queries and keys only, so values carry no position.

2.2 Value residual learning

ResFormer (Zhou et al., 2024) combines every layer's values with the first layer's, $V_n' = \lambda_{n,1} V_1 + \lambda_{n,2} V_n$, with both weights fixed at 0.5 or learned. In our indexing the first layer is layer 0. Its values $v_s^{(0)} = W_V^{(0)} \hat{x}_s^{(0)}$ are computed before any attention, from the token's input representation alone, so they are a learned, per-head encoding of the token's identity. We use the form

$$\tilde{v}_{s,g}^{(\ell)} = v_{s,g}^{(\ell)} + \lambda \, v_{s,g}^{(0)}, \qquad \ell \ge 1, \tag{2}$$

with $\lambda = 0.5$, which is the point where MGVR starts, and call it ResFormer. A second variant learns one λ per layer. Zhou et al. (2024) motivate value residuals by the loss of token-level information in deep layers and by the concentration of attention on a few tokens. In either form the weight is a constant: the same for every head, every token and every context.

2.3 Circuits that need token identity

Induction heads (Elhage et al., 2021; Olsson et al., 2022) complete patterns of the form $\dots A\, B \dots A \to B$. A previous-token head writes "preceded by A" into the residual stream at B. An induction head at the second A matches on that through its query–key circuit and copies B through its output–value circuit. The copy reads B's value, so it works only once B's identity is readable in B's value at the induction head's depth. Associative recall asks for the value stored with a key earlier in the context; multi-query associative recall (MQAR) is its standard form (Arora et al., 2023). Two-hop recall composes two lookups, $a \to b$ and then $b \to c$, across layers, and composing retrievals is a known difficulty for Transformers (Wang et al., 2024). These are the circuits we probe in Section 5.

3 Multi-head Gated Value Residuals

3.1 Definition

At every layer $\ell \ge 1$ and position s, one linear map reads the normalised attention input $\hat{x}_s$ and produces G gate logits and G^2 mixing logits,

$$(u_s, M_s) = \frac{W_z \hat{x}_s}{\sqrt{d}}, \qquad u_s \in \mathbb{R}^G, \quad M_s \in \mathbb{R}^{G \times G}, \quad W_z \in \mathbb{R}^{(G+G^2) \times d}. \tag{3}$$

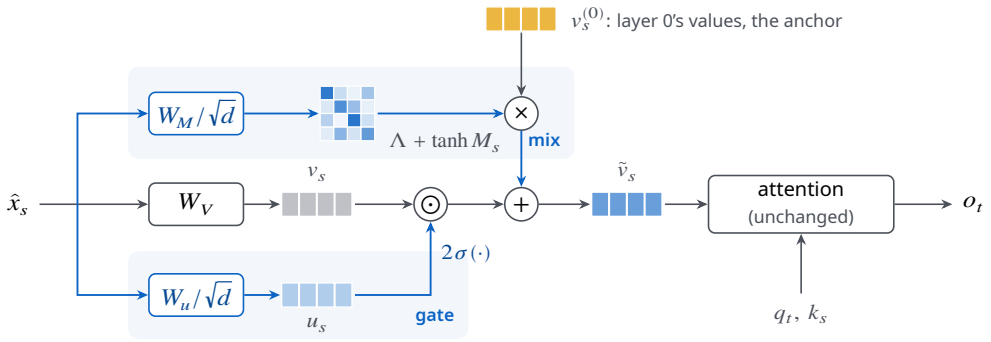


Figure 2 **One MGVR layer.** A standard attention layer computes its values v_s from its normalised input (grey). MGVR adds two small projections of the same input (blue), with $W_z = [W_u; W_M]$. The first gives a per-head gate $2\sigma(u_s)$ on the layer's own values. The second gives a token-dependent matrix $\Lambda + \tanh M_s$ that mixes layer 0's value heads, the anchor $v_s^{(0)}$, into them (Eq. (4)). Attention runs unchanged on the result $\tilde{v}_s$, which is also what the key–value cache stores.

Every key–value head g then replaces its value by

$$\tilde{v}_{s,g} = \underbrace{2\,\sigma(u_{s,g})\; v_{s,g}}_{\text{gated own value}} + \sum_{h=1}^{G} \underbrace{(\Lambda_{g,h} + \tanh M_{s,g,h})}_{\text{mixing weight}}\; v_{s,h}^{(0)}, \tag{4}$$

with a static mixing matrix $\Lambda \in \mathbb{R}^{G\times G}$ per layer. Attention runs unchanged on $\tilde{v}$. Layer 0 computes the anchor $v^{(0)}$ and passes it on without modifying its own values. Figure 2 shows one layer and Listing 1 the implementation.

3.2 Three properties

A dose per head. The gate $2\sigma(u_{s,g}) \in (0, 2)$ scales head g's own value, and row g of $\Lambda + \tanh M_s$ sets how much of each first-layer value head it adds. Heads that copy can take a large dose of identity and heads that aggregate context can take none. A head can even give up its own value in favour of identity. On its own, the gate is a head-wise, input-dependent gate on the values, one of the gating positions studied by Qiu et al. (2025).

A dose per token. The logits are computed from $\hat{x}_s$, the current state of the token whose value is formed. The dose can therefore follow both what the token is and what the layers below have made of it: identity on demand. The decision is made at the attended position, not at the query. MGVR decides what each token offers as its value, and every query that attends to it receives the same. A per-query choice would require a second attention pass; MGVR needs none.

Mixing across heads. Layer 0 spreads a token's identity over its G value heads, and the heads of later layers need not want it in the same arrangement. The mixing matrix lets head g draw on any of them. Its token-dependent part turns the added term $\sum_h \tanh(M_{s,g,h})\, v_{s,h}^{(0)}$ into a bilinear interaction, through a saturating nonlinearity, between the token's contextual state $\hat{x}_s$ and its identity $v_s^{(0)}$. A value residual adds a fixed linear term instead. Section 5.4 shows that this mixing carries most of MGVR's effect.

3.3 Initialisation

We initialise $W_z = 0$ and $\Lambda = \lambda I$. Then $2\sigma(0) = 1$ and $\tanh 0 = 0$, so Eq. (4) reduces to $\tilde{v}_{s,g} = v_{s,g} + \lambda\, v_{s,g}^{(0)}$, which is ResFormer (Eq. (2)) head by head. With $\lambda = 0$ it is standard attention. MGVR therefore starts from a known-good model and departs from it only as far as the gradients ask. Neither initialisation draws random numbers, so an MGVR model and a baseline built from the same seed share every other weight bit for bit; our controlled comparisons rely on this. We use $\lambda = 0.5$ throughout. The factor $1/\sqrt{d}$ in Eq. (3) makes the logits respond to an optimiser step independently of width. Since $\hat{x}_s$ has unit RMS, a coherent Adam step of size η on W_z moves a logit by about $\eta\sqrt{d}$ rather than ηd.

```python
class MGVR(nn.Module):
    """Multi-head Gated Value Residual for one attention layer >= 1."""
    def __init__(self, d_model, n_kv_heads, lam=0.5):
        super().__init__()
        G = self.G = n_kv_heads
        self.w_z = nn.Parameter(torch.zeros(G + G * G, d_model))  # gate + mix logits
        self.lam = nn.Parameter(lam * torch.eye(G))               # static mix
        self.scale = d_model ** -0.5

    def forward(self, x, v, v0):
        """x [B,T,d]: normalised attention input; v, v0 [B,T,G,d_h]."""
        G = self.G
        z = F.linear(x.float(), self.w_z) * self.scale                # [B,T,G+G*G]
        gate = 2 * torch.sigmoid(z[..., :G])                          # [B,T,G]
        mix = self.lam + torch.tanh(z[..., G:]).unflatten(-1, (G, G))  # [B,T,G,G]
        out = gate[..., None] * v.float() \
            + torch.einsum("btgh,bthd->btgd", mix, v0.float())
        return out.to(v.dtype)

# In the attention of layer l, before attention is computed:
#   q, k, v = project(x)                # v: [B,T,G,d_h], no rotary on v
#   if l == 0: v0 = v.float()           # the anchor, passed to every later layer
#   else:      v = mgvr(x, v, v0)
```

Listing 1 MGVR in PyTorch, as in the companion repository. The pretraining implementation computes the same arithmetic as one broadcast reduction over h, which the compiler fuses into a single kernel.

3.4 Training

W_z and Λ are trained with AdamW (Loshchilov and Hutter, 2019) without weight decay, because weight decay would pull Λ towards zero, away from the value residual. Both are kept in FP32, as are the anchor $v^{(0)}$ and the gate and mixing arithmetic.

3.5 Cost

Per layer, MGVR adds $(G + G^2)\,d + G^2$ parameters. In our pretraining testbed ($d = 512$, $G = 4$, 19 layers after layer 0) that is 194,864 parameters, 0.2% of the 96M active per token. It also costs about 1.2 MFLOP per training token (six times the parameter count), 0.2% of the model's training FLOPs. The gate and the mix add $O(G^2 d_h)$ element-wise operations per token and layer. The anchor is held through the forward pass, one value tensor per sequence, and the backward pass needs the logits and the gated values. At inference, $\tilde{v}_s$ depends on position s alone. The key–value cache therefore stores $\tilde{v}$ in place of v and does not grow, and decoding a token costs one extra $(G + G^2) \times d$ matrix–vector product per layer. Attention itself is untouched, so MGVR works unmodified with fused attention kernels (Dao et al., 2022), sliding windows and grouped-query attention. In the pretraining testbed, our compiled PyTorch implementation lengthens each training step by 4.3% (Section 4).

4 Pretraining: the same loss in fewer steps

4.1 Setup

Testbed. A 20-layer decoder with $d = 512$ and 96M parameters active per token. Attention is grouped-query, with 8 query and 4 key–value heads of dimension 64 and rotary position embeddings. Sixteen layers attend within a sliding window of 128 tokens, and four layers (every sixth and the last) attend globally over the 4,096-token context (Beltagy et al., 2020; Gemma Team, 2025). MGVR acts on layers 1–19 with $G = 4$ and $\lambda = 0.5$, which is 194,864 parameters. The feed-forward and input layers are identical in both arms and are not part of this study. Appendix A lists the remaining settings.

Table 1 Held-out loss (nats/token) of the two arms of the pretraining comparison, which share the initial weights, the data order and every hyperparameter. *Fewer steps*: $1 - s/s_T$, where s_T is the step at which the Transformer's stable-phase curve reaches MGVR's loss at step s. *Effective compute*: s_T/s divided by MGVR's relative step time, 1.043 (260.2K vs 249.5K tokens/s on one RTX PRO 6000). †From step 650 on, MGVR is below the best loss the Transformer reaches before its learning-rate decay. Shaded rows: the decay.

Step	Transformer	MGVR	Δ	Fewer steps	Effective compute
100	5.4132	**5.3859**	−0.0273	2%	0.98×
200	4.4977	**4.1492**	−0.3484	17%	1.16×
300	3.7605	**3.5572**	−0.2033	20%	1.20×
400	3.5152	**3.3944**	−0.1207	17%	1.15×
500	3.3799	**3.2535**	−0.1264	19%	1.18×
600	3.2704	**3.1807**	−0.0896	20%	1.20×
700	3.2080	**3.1149**	−0.0931	> best†	—
800	3.0459	**2.9800**	−0.0659	—	—
900	2.9049	**2.8512**	−0.0537	—	—
950	2.8624	**2.8124**	−0.0500	—	—

Data. A 1.07B-token sample of a multilingual pretraining mixture. It contains web text, reference and educational text, code and mathematics, and a small share of dialogue, in English, Chinese, German, Japanese and Russian. Training reads random 4,096-token windows. The held-out loss is measured on the same 2.1M tokens of a held-out split at every evaluation.

Training. 954 steps of 2^{20} tokens (1.0B tokens) on a warmup–stable–decay schedule (Hu et al., 2024; Hägele et al., 2024): 100 warmup steps, a constant learning rate, and a linear decay to zero over the last 20% of steps, from step 763. Muon (Jordan et al., 2024b; Liu et al., 2025) trains the matrices with learning rate 0.02. AdamW trains the remaining parameters with learning rate $3 \cdot 10^{-3}$ and $\beta = (0.9, 0.95)$. Weight decay is 0.1 and gradients are clipped at norm 1.0. MGVR's parameters join the AdamW group without weight decay (Section 3).

Protocol. The two arms are a Transformer and the same model with MGVR. They start from identical weights, because MGVR draws no random numbers. They read identical batches in identical order and share every hyperparameter, so the comparison is paired. Each arm runs on one RTX PRO 6000 GPU, with the held-out loss evaluated every 50 steps. We fixed the decision rule before training: MGVR passes if its held-out loss is at least 0.005 nats below the Transformer's at every evaluation in the last quarter of training.

Steps to loss. For each evaluation of MGVR at step s during the stable phase, we find the step s_T at which the Transformer reaches the same loss, interpolating linearly between its evaluations. MGVR saves the fraction $1 - s/s_T$ of the steps. We divide s_T/s by MGVR's relative step time to get its *effective compute*: the factor of wall-clock compute the Transformer would need to match it. We read steps to loss only in the stable phase, where both arms have the same learning rate at every step. The decay changes the shape of the curves, so there we compare the arms at equal steps.

4.2 Results

Figure 1 and Table 1 show the comparison. MGVR pulls ahead as warmup ends, and by step 200 it is 0.35 nats below the Transformer. From then on its lead holds steady in the measure that matters. At every evaluation from step 200 to step 600, MGVR reaches the Transformer's loss with 16–22% fewer steps; equivalently, the Transformer needs 1.20–1.28× as many. The loss gap narrows as both curves flatten, from 0.35 nats at step 200 to 0.09 at step 600, but the step advantage does not. At step 600 MGVR matches the best loss the Transformer reaches before its learning-rate decay, at step 750, and it stays below that loss from step 650 on. MGVR is ahead at every one of the 19 evaluations.

The decay preserves the lead. Both arms drop by about 0.3 nats during the decay, and MGVR ends at 2.812, against 2.862 for the Transformer: 0.050 nats lower. At every evaluation in the last quarter, MGVR is 0.050–0.074 nats ahead, 10 to 15 times the margin the decision rule required.

Cost. The Transformer trains at 260.2K tokens/s and MGVR at 249.5K, so an MGVR step takes 4.3% longer. The arithmetic MGVR adds is 0.2% of the model's FLOPs. The rest of the overhead is memory traffic for the anchor, the logits and the gated values. Charged its full measured cost, MGVR delivers 1.15–1.23× effective compute from step 200 to step 600. It breaks even in wall-clock time at a step saving of 4.1%, about a quarter of the smallest saving it achieves in that range.

5 Mechanism: identity where circuits need it

Pretraining shows that MGVR learns faster, but not what it learns faster. We isolate that on synthetic tasks that need token identity at depth, using a minimal model in which the value path is the only difference between arms.

5.1 Setup

Model. A decoder with pre-norm RMSNorm blocks, rotary position embeddings, causal multi-head attention with four heads ($G = H = 4$, $d = 128$), a GELU MLP of width $4d$ and untied embeddings.

Arms. Six variants differ only in how layers $\ell \geq 1$ form their values:

- the Transformer;
- ResFormer (Eq. (2), $\lambda = 0.5$), and ResFormer with a learned λ per layer;
- MGVR;
- two ablations of MGVR: the gate alone, $\tilde{v} = 2\sigma(u)\, v + \lambda v^{(0)}$, and the mix alone, $\tilde{v} = v + (\Lambda + \tanh M)\, v^{(0)}$.

All MGVR variants start exactly as ResFormer.

Controls. For each seed, every arm sees the same training batches, the same held-out batch and the same initial weights. Training uses AdamW with $\beta = (0.9, 0.95)$ and weight decay 0.1. Decay applies to the matrices, but not to the embeddings or MGVR's parameters. The schedule is 100 warmup steps and a cosine decay to 10% of the peak learning rate, with batch 64 and BF16 autocast. Every cell is run with three seeds, and we report means and standard deviations.

Metrics. Accuracy is measured on a fixed held-out batch of 256 sequences, at the positions whose target the context determines. AUC is the mean held-out accuracy over all evaluations of a run, so it measures learning speed. For induction we also record each head's *induction score*: the mean attention from each repeated token to the token that followed its earlier occurrence, i.e. the induction pattern of Olsson et al. (2022). We report the best head's score.

5.2 Induction heads

Task. A sequence of 129 random tokens from a vocabulary of 512, followed by its repeat, 256 tokens in all. The model is scored on the repeat, where every next token is determined by the first occurrence. We use two layers, the smallest model that can form the two-head induction circuit. To time the circuit's formation precisely, we run each learning rate (10^{-4}, $3 \cdot 10^{-4}$ and 10^{-3}) as its own experiment of 600 steps, with an evaluation every 5 steps.

MGVR learns to copy sooner. MGVR learns the task faster at every learning rate, and the three architectures keep the same order everywhere (Table 2): the Transformer, then ResFormer, then MGVR. Formation is slowest, and therefore timed most precisely, at $\eta = 10^{-4}$. There MGVR reaches 90% accuracy 14% sooner than the Transformer (405 against 473 steps) and 4% sooner than ResFormer.

Table 2 Induction (two layers, repeated random tokens), mean and standard deviation over three seeds. Left: steps until held-out copy accuracy first reaches 90%, at three fixed learning rates. Right, at $\eta = 10^{-4}$: the lag from the moment the best head's induction score reaches 0.3 to 90% accuracy, and the final induction score.

	Steps to 90% accuracy			At $\eta = 10^{-4}$	
Arm	$\eta = 10^{-4}$	$3 \cdot 10^{-4}$	10^{-3}	Pattern-to-copy lag	Induction score
Transformer	473 ±13	188 ±3	113 ±3	133 ±12	0.678 ±0.015
ResFormer	422 ±8	182 ±3	108 ±3	92 ±6	0.722 ±0.012
MGVR	**405** ±5	**178** ±3	**105** ±0	**80** ±5	**0.736** ±0.012

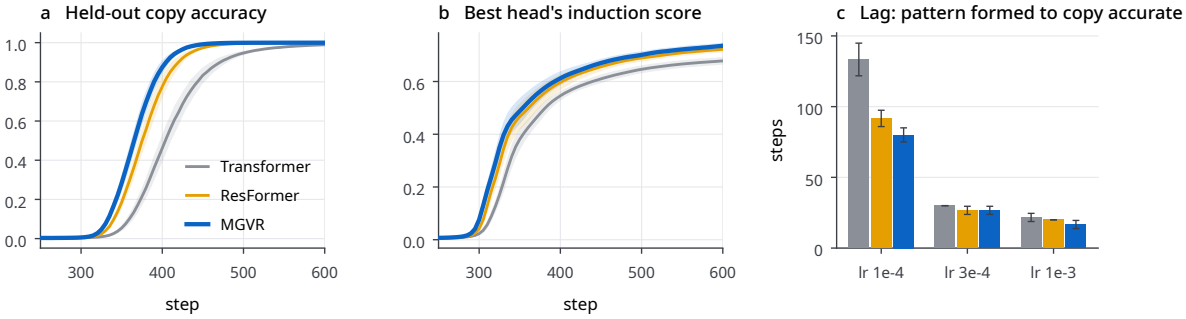


Figure 3 **MGVR shortens the step from pattern to copy.** Two-layer induction, three seeds (mean and standard deviation). **a** Held-out copy accuracy and **b** the best head's induction score, both at $\eta = 10^{-4}$. The pattern forms at nearly the same time in all three arms, and MGVR turns it into accurate copying fastest. **c** Steps from an induction score of 0.3 to 90% accuracy, at each learning rate.

The gain is in the copy, not the match. The timing shows where the gain comes from (Fig. 3). With MGVR the induction attention pattern forms only slightly earlier: the best head's score passes 0.3 at step 325, against 340 for the Transformer. What shrinks is the lag from the pattern to an accurate copy. It is 133 steps for the Transformer, 92 for ResFormer and 80 for MGVR, which is 40% shorter than the Transformer's. This is the signature of a change to the output–value circuit. When the induction head attends to the right token, the value it reads already carries that token's identity from layer 0, so the copy becomes accurate as soon as the pattern forms. The query–key circuit that finds the token is unchanged. The lag is shorter than the Transformer's at every learning rate. The final pattern is also sharper, with an induction score of 0.736, against 0.722 for ResFormer and 0.678 for the Transformer.

5.3 Composing two lookups

Task. Two-hop associative recall. The context lists $2n$ shuffled pairs that encode n chains, $a \to b$ and $b \to c$, with a, b and c drawn from disjoint thirds of the vocabulary. The model is then queried with every a, in random order, and must produce its c. Each answer needs two retrievals composed across layers: b from a, then c from b. We use four layers and $n = 10$ or $n = 20$ chains (60 and 120 tokens), and train for 6,000 steps at learning rates 10^{-3} and $3 \cdot 10^{-3}$. We report each arm at its better rate by AUC.

Results. This is where MGVR has its largest effect (Table 3 and Fig. 4). It learns 25% faster than the Transformer with 10 chains and 49% faster with 20 (AUC 0.273 against 0.219, and 0.167 against 0.112), and 7% and 18% faster than ResFormer. The gain grows with the size of the problem. MGVR is also far more consistent: its standard deviation across seeds is four to ten times smaller than that of the Transformer or ResFormer. Within 6,000 steps no arm masters the task, and final accuracies range from 16% to 30%. The comparison is therefore one of learning speed, which is what AUC measures. MGVR also ends highest, at 30.4% and 19.9%.

Table 3 Two-hop associative recall (four layers), mean and standard deviation over three seeds at each arm's best learning rate. AUC: held-out accuracy averaged over the 6,000 training steps (learning speed); final: accuracy over the last three evaluations (%). Bold: within $5 \cdot 10^{-4}$ of the best.

	10 chains		20 chains	
Arm	AUC	Final (%)	AUC	Final (%)
Transformer	0.219 ±0.008	28.2 ±1.1	0.112 ±0.008	15.8 ±1.2
ResFormer	0.254 ±0.017	30.1 ±1.5	0.141 ±0.011	18.5 ±1.2
MGVR, mix only	0.272 ±0.001	30.0 ±0.3	**0.167 ±0.008**	19.5 ±0.6
MGVR	**0.273 ±0.002**	**30.4 ±0.0**	**0.167 ±0.002**	**19.9 ±0.2**

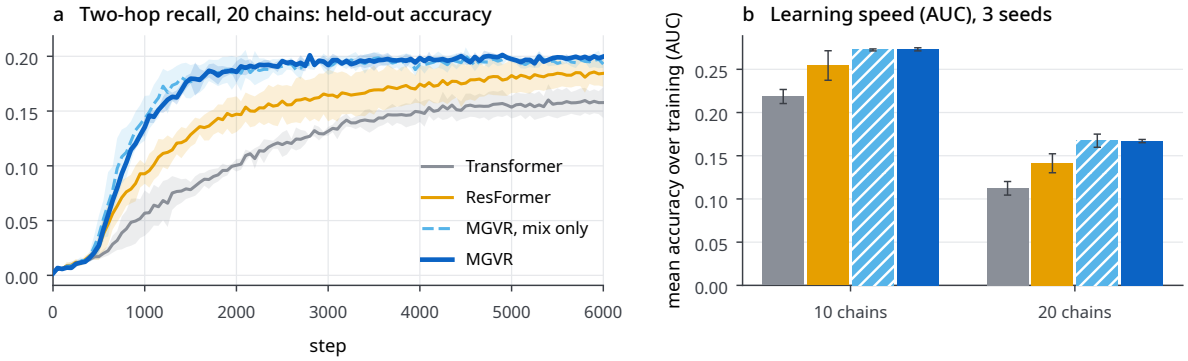


Figure 4 **MGVR learns to compose lookups faster.** Two-hop associative recall with four layers, three seeds, each arm at its better learning rate. **a** Held-out accuracy with 20 chains (mean and standard deviation). **b** Learning speed (AUC: mean held-out accuracy over training) with 10 and 20 chains. The mix alone (hatched) matches full MGVR.

5.4 What does the work

The ablations separate MGVR's two parts (Table 3; all six arms in Appendix D). On two-hop recall the mix alone matches the full method, with AUC 0.272 and 0.167 against 0.273 and 0.167. A learned residual weight does not close the gap: ResFormer with a learned λ reaches 0.257 and 0.144. What helps composition is therefore not how much identity a layer receives. It is that the dose depends on the token and can be routed across heads. On induction both parts contribute. The gate alone and the mix alone each match or beat ResFormer at every learning rate. Together they are fastest at $\eta = 10^{-4}$ and tie the mix alone at the higher rates (Table 7).

6 Related work

Value residuals. ResFormer (Zhou et al., 2024) introduced residual connections from the values of the first layer to those of every later layer, with fixed or learned weights. Its variant SVFormer shares the first layer's values outright, which nearly halves the key–value cache. The NanoGPT speedrun (Jordan et al., 2024a) mixes extra embeddings into the values of its attention layers, inspired by value residuals. MGVR keeps the first layer's values as the source of identity and makes the weights depend on the token and the head.

Dense and dynamic cross-layer connections. DenseFormer (Pagliardini et al., 2024) lets every block read a learned weighted average of all earlier block outputs. Hyper-Connections (Zhu et al., 2024) replace the residual connection with several streams joined by learnable, optionally input-dependent weights. MUDDFormer (Xiao et al., 2025) generates per-position weights that aggregate earlier layers separately into each block's query, key, value and residual inputs. MGVR is a narrow member of this family. It connects one source, layer 0, to one destination, the values, at the granularity of heads, and leaves the residual stream untouched, for 0.2% of the active parameters. Cross-Layer Attention (Brandon et al.,

2024) shares keys and values between adjacent layers to shrink the cache. MGVR keeps per-layer values and the cache size.

Gating in attention. Qiu et al. (2025) study sigmoid gates at several positions in the attention layer. They find a head-specific gate on the attention output most effective, and head-wise gates on the values also improve on ungated attention. MGVR's gate sits on the values, but its role differs: together with the mix, it trades a head's own value against identity, token by token. Gated skip paths go back to highway networks (Srivastava et al., 2015). MGVR gates an identity path of a different kind: the token's own first-layer encoding, delivered through attention.

Circuits and recall. Induction heads (Elhage et al., 2021; Olsson et al., 2022) are the canonical circuit for in-context copying and form in a sudden transition during training. MQAR (Arora et al., 2023) measures in-context recall, and composing retrievals is a known difficulty for Transformers (Wang et al., 2024). We use these tasks to probe where token identity is needed. Supplying it on demand shortens the copy phase of induction-head formation and speeds up composition.

7 Conclusion

MGVR turns the value residual from a constant into a decision that every layer makes per token and per head: how much of a token's identity to carry in its value, and from which of the first layer's heads. The change is small, at 0.2% of the active parameters with attention left as it is, and it starts exactly where a value-residual Transformer does. It pays off. In a controlled pretraining comparison, MGVR reaches the same loss in 16–22% fewer steps, worth 1.15–1.23× effective compute after its cost, and ends at a lower loss. In a mechanism study, it makes induction heads copy accurately sooner and composes lookups faster, and its token-dependent mixing of value heads drives the gains. MGVR is a drop-in change for any multi-head or grouped-query attention layer, and it is the default in our models.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, 2024.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang,

Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, et al. modded-nanogpt: Speedrunning the NanoGPT baseline. https://github.com/KellerJordan/modded-nanogpt, 2024a.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024b.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. arXiv:2209.11895.

Matteo Pagliardini, Amirkeivan Mohtashami, François Fleuret, and Martin Jaggi. DenseFormer: Enhancing information flow in transformers via depth weighted averaging. *arXiv preprint arXiv:2402.02622*, 2024.

Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. *arXiv preprint arXiv:2505.06708*, 2025.

Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

Boshi Wang, Xiang Yue, Yu Su, and Huan Sun. Grokked transformers are implicit reasoners: A mechanistic journey to the edge of generalization. In *Advances in Neural Information Processing Systems*, 2024.

Da Xiao, Qingye Meng, Shengping Li, and Xingyuan Yuan. MUDDFormer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections. In *International Conference on Machine Learning*, 2025.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019.

Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. *arXiv preprint arXiv:2410.17897*, 2024.

Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. *arXiv preprint arXiv:2409.19606*, 2024.

A Pretraining details

Table 4 lists the pretraining settings. Both arms are built from the same seed. MGVR's parameters are initialised deterministically ($W_z = 0$, $\Lambda = 0.5\,I$), so every other weight of the two models is identical at step 0. Both arms also read the same batches in the same order.

Table 4 Pretraining comparison: settings shared by both arms, and MGVR's. The feed-forward and input layers are identical in both arms.

Model	
Layers, width	20, $d = 512$, pre-norm (RMSNorm)
Attention	grouped-query, 8 query and 4 key–value heads of dimension 64
Attention span	sliding window of 128 in 16 layers; global in layers 5, 11, 17 and 19
Positions, context	rotary (base $5 \cdot 10^5$), 4,096 tokens
Vocabulary	65,536 tokens, untied input and output embeddings
Active parameters	96M per token
MGVR	layers 1–19, $G = 4$, $\lambda = 0.5$; 194,864 parameters
Training	
Tokens	954 steps × 2^{20} tokens = 1.0B
Schedule	warmup–stable–decay: 100 warmup steps, linear decay to 0 over the last 20%
Matrices	Muon, learning rate 0.02
Other parameters	AdamW, learning rate $3 \cdot 10^{-3}$, $\beta = (0.9, 0.95)$
MGVR parameters	AdamW, learning rate $3 \cdot 10^{-3}$, no weight decay, FP32
Weight decay, clipping	0.1; gradient norm 1.0
Precision	BF16 with FP8 matrix multiplications; MGVR arithmetic in FP32
Evaluation	
Held-out loss	2.1M tokens of a held-out split (the same rows each time), every 50 steps
Hardware	one RTX PRO 6000 per arm

One run per arm. Each arm was trained once. Two features make a single paired run informative. First, the arms share their initial weights and their data order, which removes the two largest sources of run-to-run variation from the difference between them. Second, the difference has the same sign at all 19 evaluations, and in the last quarter of training it is 10 to 15 times the decision threshold fixed before training. The synthetic study (Section 5) uses three seeds per cell throughout.

B Synthetic study details

Table 5 lists the task settings. Token 0 is a separator that begins every sequence and is never scored. Sequences shorter than the context are padded with random tokens.

- **Induction.** $L = T/2 + 1$ random tokens, then the same L tokens again, cut to T inputs. The scored positions are the repeat, where each target is the token after the current token's earlier occurrence. The induction score of a head is its mean attention from each repeat position t to position $t - L + 1$. The previous-token score, its mean attention from t to $t - 1$, is logged as well.
- **Two-hop recall.** n chains $a \to b \to c$ with a, b and c from disjoint thirds of the vocabulary, and a and b distinct within a sequence. The $2n$ pairs $(a\,b)$ and $(b\,c)$ appear in random order, followed by the n queries a in a new random order, each followed by its c. The scored positions are the queries.
- **MQAR** (Arora et al., 2023). n key–value pairs, with distinct keys from the lower half of the vocabulary and values from the upper half, followed by the n keys in random order, each followed by its value. The scored positions are the queries.

Learning rates are selected per experiment and arm. Every arm runs every rate in its grid, and the rate with the best mean AUC over the three seeds is reported, so no arm receives more tuning than another. Induction is in addition reported at each of three fixed learning rates. The models have 0.5M–0.9M

parameters, and several runs share one GPU at a time.

Table 5 Synthetic tasks (main grid). Every arm runs every listed learning rate with three seeds. Each induction learning rate is its own experiment; for two-hop recall and MQAR each arm is reported at its best rate by mean AUC. Batch 64, held-out batch 256, loss on the scored positions, 100 warmup steps, cosine decay to 10%.

Task	Layers	d	Vocabulary	Length	Steps	Eval. every	Learning rates
Induction	2	128	512	256	600	5	10^{-4}, $3 \cdot 10^{-4}$, 10^{-3}
Two-hop, 10 chains	4	128	512	60	6,000	50	10^{-3}, $3 \cdot 10^{-3}$
Two-hop, 20 chains	4	128	512	120	6,000	50	10^{-3}, $3 \cdot 10^{-3}$
MQAR, 16 pairs	2	128	512	64	12,000	100	10^{-3}, $3 \cdot 10^{-3}$

C Calibration of the synthetic tasks

The main grid follows two earlier grids, and the repository contains all three. The first grid used 2,000 steps of induction, 3,000 steps of MQAR with 64 pairs and 4,000 steps of two-hop recall with 40 chains, each at learning rates $5 \cdot 10^{-4}$, $1.5 \cdot 10^{-3}$ and $4.5 \cdot 10^{-3}$. Its tasks were too easy or too hard to separate the arms. At the best learning rate, induction reached 90% accuracy within 120 steps in every arm, with two or four layers (AUC 0.943–0.952). MQAR stayed below 9% accuracy in every arm. Two-hop recall reached only 6% for the Transformer, so we stopped that experiment before running the other arms.

Table 6 Calibration grid (one seed, Transformer and MGVR only), run to choose task sizes and budgets where the Transformer needs a substantial part of the budget. AUC and final held-out accuracy (%).

	Transformer		MGVR	
Setting	AUC	Final	AUC	Final
Induction, V=4096, d=64, batch 8, η=10^{-3}	0.000	0.0	0.000	0.0
Induction, V=4096, d=64, batch 8, η=$3 \cdot 10^{-4}$	0.000	0.0	0.000	0.0
MQAR, 16 pairs, 4,000 steps	0.173	19.8	0.172	19.9
MQAR, 32 pairs, 4,000 steps	0.087	10.7	0.083	13.1
MQAR, 32 pairs, η=$3 \cdot 10^{-3}$	0.039	6.0	0.045	6.5
Two-hop, 10 chains, 6,000 steps	0.216	28.9	0.272	30.4
Two-hop, 20 chains, 6,000 steps	0.111	16.2	0.165	20.1

A calibration grid with one seed, the Transformer and MGVR then bracketed the difficulty (Table 6). Induction with a 4,096-token vocabulary, $d = 64$ and batch 8 did not form within 1,500 steps. MQAR with 16 or 32 pairs learned slowly. Two-hop recall learned partway in both sizes, already with the gap reported in Section 5.3. The main grid keeps the settings in which the Transformer needs a substantial part of the budget. It runs induction in the original setting at three fixed learning rates with an evaluation every 5 steps, so that formation takes hundreds of steps and its timing is resolved. It also gives MQAR three times the steps.

D Full synthetic results

Tables 7 and 8 report all six arms.

The gate alone. On induction the gate alone is faster than ResFormer at the two lower learning rates and ties it at the highest. On two-hop recall it falls below ResFormer (AUC 0.243 against 0.254 with 10 chains, 0.133 against 0.141 with 20). A gate on the layer's own values does not help composition without the token-dependent mix, which is the part MGVR relies on there (Section 5.4).

MQAR. With 16 pairs and two layers, MQAR outcomes depend strongly on the seed. At the higher learning rate some seeds solve the task within 12,000 steps, while others stay near the 20% accuracy

that every arm reaches early. MGVR reaches 90% accuracy with two of three seeds and has the highest mean AUC (0.495). The mix alone, ResFormer with a learned λ, and the Transformer reach it with one seed each, and the Transformer's seed is the fastest of all, at step 1,500. ResFormer and the gate alone reach it with none. With outcomes this dispersed, three seeds do not separate the arms, and we report MQAR for completeness.

Table 7 Induction, all six arms at the three fixed learning rates (three seeds). Steps until held-out copy accuracy reaches 50% and 90%, and until the best head's induction score reaches 0.3 and 0.5 (evaluations every 5 steps); final induction score after 600 steps.

Arm	Acc. ≥ 50%	Acc. ≥ 90%	Score ≥ 0.3	Score ≥ 0.5	Final score
$\eta = 0.0001$					
Transformer	407 ±8	473 ±13	340 ±5	385 ±9	0.678 ±0.015
ResFormer	378 ±6	422 ±8	330 ±5	363 ±15	0.722 ±0.012
ResFormer (learned λ)	378 ±6	417 ±8	330 ±5	362 ±13	0.724 ±0.011
MGVR, gate only	373 ±6	415 ±5	325 ±5	358 ±15	0.726 ±0.012
MGVR, mix only	372 ±3	412 ±8	328 ±3	358 ±15	0.731 ±0.011
MGVR	368 ±3	405 ±5	325 ±5	355 ±15	0.736 ±0.012
$\eta = 0.0003$					
Transformer	178 ±3	188 ±3	158 ±3	163 ±3	0.983 ±0.001
ResFormer	168 ±3	182 ±3	155 ±0	160 ±0	0.998 ±0.000
ResFormer (learned λ)	168 ±3	180 ±0	153 ±3	160 ±0	0.999 ±0.000
MGVR, gate only	168 ±3	180 ±0	153 ±3	160 ±0	0.998 ±0.000
MGVR, mix only	168 ±3	178 ±3	153 ±3	160 ±0	0.988 ±0.002
MGVR	168 ±3	178 ±3	152 ±3	158 ±3	0.988 ±0.001
$\eta = 0.001$					
Transformer	103 ±3	113 ±3	92 ±3	93 ±3	0.991 ±0.001
ResFormer	100 ±0	108 ±3	88 ±3	92 ±3	0.998 ±0.000
ResFormer (learned λ)	100 ±0	108 ±3	88 ±3	92 ±3	0.998 ±0.000
MGVR, gate only	100 ±0	108 ±3	88 ±3	92 ±3	0.998 ±0.000
MGVR, mix only	98 ±3	105 ±0	88 ±3	92 ±3	0.996 ±0.001
MGVR	98 ±3	105 ±0	88 ±3	90 ±0	0.996 ±0.001

Table 8 Two-hop recall and multi-query associative recall (MQAR, 16 pairs, two layers, 12,000 steps), all six arms at each arm's best learning rate (three seeds). For MQAR: the seeds whose held-out accuracy reached 90%, and the step at which they did.

	Two-hop, 10 chains		Two-hop, 20 chains		MQAR, 16 pairs		
Arm	AUC	Final (%)	AUC	Final (%)	AUC	Solved	Step ≥ 90%
Transformer	0.219 ±0.008	28.2 ±1.1	0.112 ±0.008	15.8 ±1.2	0.414 ±0.417	1/3	1500 ±0
ResFormer	0.254 ±0.017	30.1 ±1.5	0.141 ±0.011	18.5 ±1.2	0.182 ±0.003	0/3	—
ResFormer (learned λ)	0.257 ±0.013	30.2 ±0.4	0.144 ±0.020	17.9 ±2.5	0.256 ±0.055	1/3	12000 ±0
MGVR, gate only	0.243 ±0.017	29.8 ±1.2	0.133 ±0.004	18.1 ±0.6	0.184 ±0.005	0/3	—
MGVR, mix only	0.272 ±0.001	30.0 ±0.3	0.167 ±0.008	19.5 ±0.6	0.363 ±0.201	1/3	6500 ±0
MGVR	0.273 ±0.002	30.4 ±0.0	0.167 ±0.002	19.9 ±0.2	0.495 ±0.256	2/3	5500 ±1556

E Verification

The harness ships with checks that pass (`python -m mgvr.test_harness`, CPU, about a minute):

- **Formula.** The module computes Eq. (4), checked one token and one head at a time against a direct evaluation with random W_z and Λ, for MGVR and both ablations.

- **Initialisation.** At initialisation every MGVR variant reproduces ResFormer's logits bit for bit, and with $\lambda = 0$ the Transformer's. All arms share every other weight for a given seed.
- **Attention paths.** The analysis path, with explicit attention probabilities, reproduces the training path's logits. Attention rows sum to one and never look ahead.
- **Tasks.** Every scored target is determined by its context, masks and shapes agree, and the separator is never scored.
- **Training.** A short run of every task trains, logs and summarises.

The reference module and the implementation used for pretraining agree to $9 \cdot 10^{-16}$ in float64 on random inputs.